Sistema de Gestión de Emergencias Médicas con Concurrencia: diseño, implementación y evaluación en Java

Edwin Alexander Aguirre Monsalve

Institución Universitaria Digital de Antioquia (IUDigital)

S30 – EA2: Programación concurrente

Yeiler Alberto Quintero Barco

27 de octubre de 2026

Resumen

Se diseñó y evaluó un simulador de emergencias médicas con concurrencia en Java, conforme al caso de estudio y la guía S30–EA2. El prototipo recibe llamadas mediante operadores concurrentes, prioriza incidentes por gravedad, espera y distancia, y reserva de manera atómica una ambulancia, un médico y un equipo. Un componente asíncrono comunica eventos a un hospital simulado, mientras otro muestra el estado de la operación. Se analizaron fallos, reintentos y cargas de 20, 60 y 120 llamadas. El registro de carga de 120 incidentes reportó 120 completados y ninguna falla; las ocho pruebas automatizadas también terminaron correctamente. Se trata de una simulación académica con coordenadas y tiempos sintéticos, no de un sistema clínico validado.

Palabras clave: concurrencia, Java, sincronización, emergencias médicas, recursos compartidos.

Introducción

El caso de estudio propone coordinar emergencias simultáneas con ambulancias, médicos y equipamiento limitados. En ese contexto, la ejecución paralela facilita la recepción y el traslado, pero puede generar condiciones de carrera o esperas indebidas si el estado compartido carece de sincronización (Caso de estudio, s. f.). La actividad exige modelar operadores, despachadores y recursos concurrentes, comunicar subsistemas y evaluar su comportamiento (S30–EA2, s. f.).

Objetivo y alcance

El objetivo es implementar y documentar una simulación reproducible en consola, con priorización dinámica, reserva indivisible de recursos, monitoreo y recuperación de averías. El sistema no gestiona datos de pacientes, geolocalización real ni comunicaciones con servicios de salud externos. Se documentan pruebas, limitaciones y decisiones de diseño para valorar el comportamiento del prototipo.

Correspondencia con los requisitos

Tabla 1

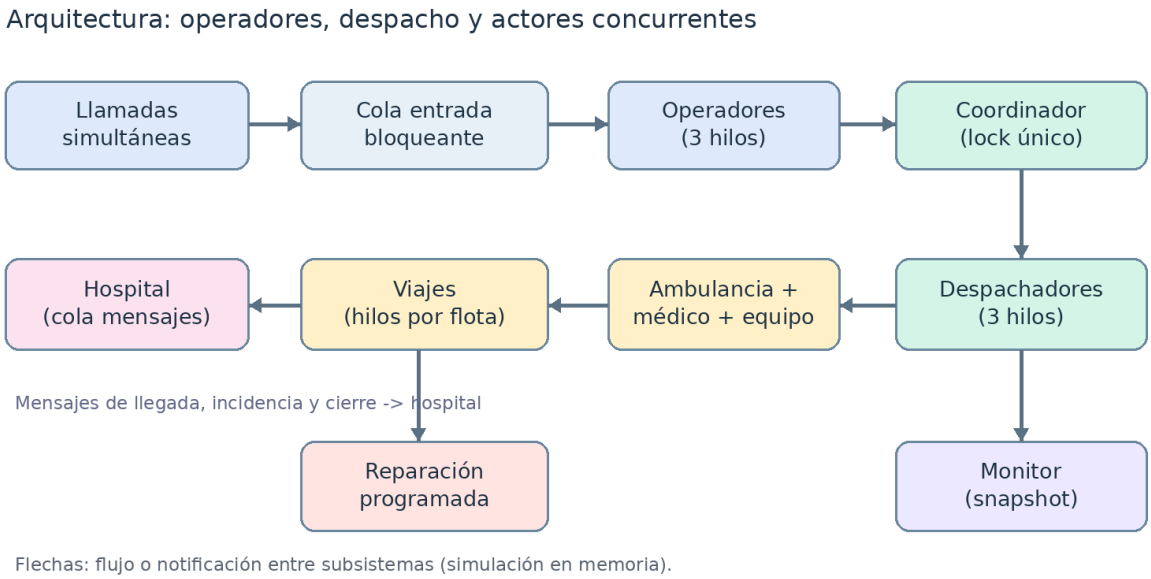
Trazabilidad entre las exigencias y las funciones implementadas

Requisito	Mecanismo y evidencia
Llamadas simultáneas	Cola de ingreso acotada y operadores en un pool de hilos.
Recursos compartidos	Reserva conjunta de ambulancia, médico y equipo bajo un único candado.
Prioridad multivariable	Puntaje por gravedad, tiempo de espera y distancia Haversine.
Comunicación y monitoreo	Cola de mensajes al hospital y monitor periódico de estado.
Fallos y rendimiento	Reintentos, reparación y pruebas con 20, 60 y 120 llamadas.

Arquitectura de componentes

Figura 1

Flujo concurrente de recepción, despacho y recuperación

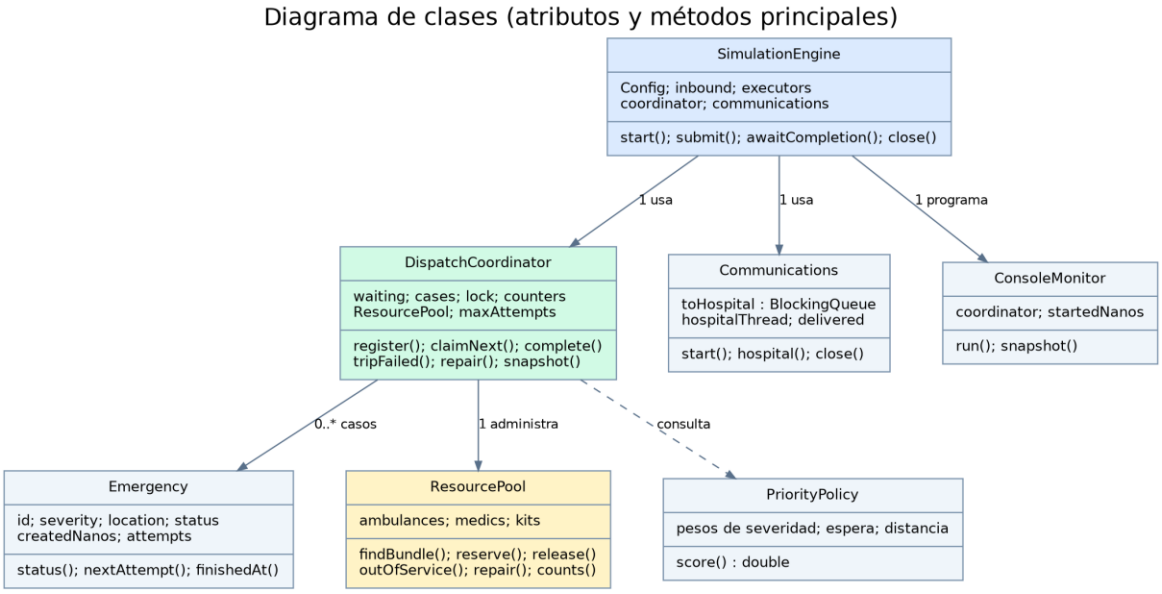


Nota. Diagrama del proyecto. Las flechas representan intercambios de trabajo o mensajes entre componentes. La cola ArrayBlockingQueue (250 elementos) desacopla la recepción del registro. Los operadores registran los casos; el coordinador decide el despacho; los trabajadores de ambulancia ejecutan el traslado y notifican al hospital. La supervisión obtiene instantáneas coherentes y programa reparaciones sin mantener ocupado el candado de reservas.

Modelo de clases y responsabilidades

Figura 2

Clases centrales y relaciones funcionales



Nota. Diagrama simplificado de las entidades implementadas. Se conservan los nombres del código Java.

Emergency representa cada incidente y almacena identificación, gravedad, coordenadas, estado y número de intentos. GeoPoint valida coordenadas y calcula distancias mediante Haversine; PriorityPolicy calcula el puntaje en función de gravedad, espera y kilómetros. ResourcePool agrupa ambulancias, médicos y equipos, incluyendo el estado y la asignación vigente de cada recurso.

DispatchCoordinator protege la cola de espera, selecciona incidentes y ejecuta la reserva atómica. SimulationEngine crea los pools de operadores, despachadores y trabajadores de ambulancia, recibe solicitudes y coordina su cierre. Communications procesa mensajes dirigidos al hospital mediante un consumidor independiente; ConsoleMonitor imprime estados y métricas desde una instantánea del coordinador. Main permite seleccionar escenarios y configurar parámetros.

Todas las entidades operan en memoria. El proyecto incluye las pruebas en ProjectTests.java y scripts de compilación y ejecución para Windows y sistemas Unix.

Concurrencia, sincronización y comunicación

Procesamiento concurrente y colas seguras

SimulationEngine separa el trabajo en tres operadores y tres despachadores por defecto, más un pool de trabajadores para las ambulancias. Los operadores consumen ArrayBlockingQueue; el hospital recibe eventos mediante LinkedBlockingQueue y un hilo propio. ScheduledExecutorService programa actualizaciones de monitoreo y la recuperación de ambulancias averiadas. ExecutorService delimita el número de trabajadores y facilita gestionar el ciclo de vida de la simulación.

Reserva indivisible y prevención de condiciones de carrera

El método claimNext de DispatchCoordinator obtiene el candado lock antes de examinar incidentes pendientes, calcular puntuaciones, elegir un paquete de recursos y reservarlo. ResourcePool.reserve verifica la disponibilidad de los tres elementos y cambia sus estados dentro de la misma sección crítica: si falta uno, no se asigna ningún paquete. La liberación y la reparación también se realizan bajo lock y notifican a los despachadores con notifyAll. La prueba de exclusión mutua ejecuta 12 solicitudes de despacho sobre una única ambulancia y verifica una sola asignación activa.

Interbloqueos, observación y cierre

No se adquieren varios candados de aplicación de forma anidada. Las esperas simuladas del trayecto, los mensajes al hospital y la impresión del monitor ocurren fuera de la sección crítica. Cuando no hay recursos suficientes, claimNext espera con límite temporal y vuelve a examinar la cola. El método snapshot copia los contadores y estados bajo el candado, por lo que cada lectura del monitor es coherente, aunque las tareas continúan avanzando.

Priorización y tolerancia a fallos

La política académica es P = G + 15T − 10D, donde G representa la gravedad (crítica: 1000; grave: 700; moderada: 400; leve: 100), T la espera en segundos y D la distancia en kilómetros. Los pesos son didácticos y no constituyen triaje validado. La distancia se estima mediante Haversine; no se calcula sobre vías reales.

Tabla 2

Ejemplo de puntuación para incidentes ubicados a 2 km

Caso	Gravedad	Espera	Puntaje
A	Crítica	0 s	980
B	Grave	10 s	830
C	Leve	70 s	1130

El coordinador recalcula la prioridad de los pendientes en cada decisión en vez de utilizar una cola de prioridad cuyo comparador dependa continuamente del reloj. El envejecimiento aumenta el puntaje de los casos que esperan; reduce el riesgo de postergación, aunque una prueba de puntaje no demuestra formalmente ausencia de inanición bajo cualquier carga.

Simulación de averías y reintentos

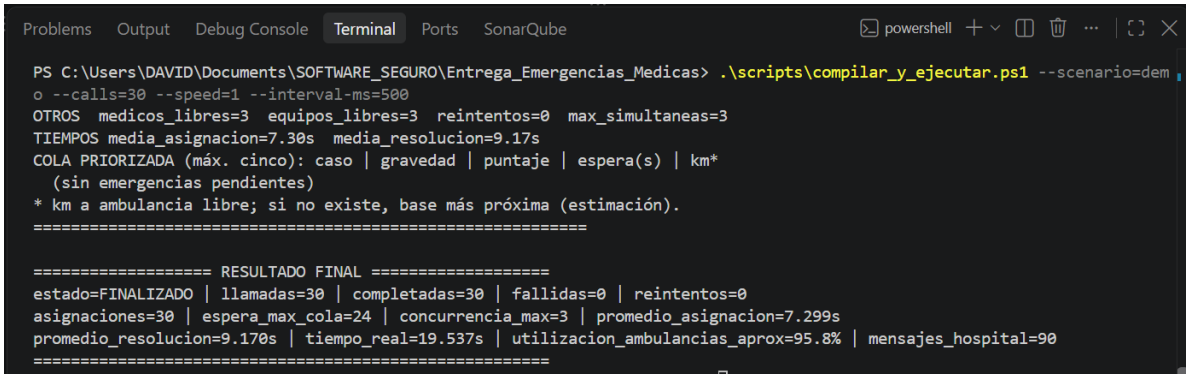
Durante un traslado puede fallar una ambulancia. Entonces pasa a FUERA_DE_SERVICIO, libera al médico y al equipo y el incidente vuelve a EN_ESPERA si quedan intentos. La reparación programada devuelve la ambulancia a DISPONIBLE. Si se alcanza el máximo configurado —tres en los escenarios habituales— el incidente termina en FALLIDA. El hospital recibe el aviso por la cola de mensajes. El escenario failure y una prueba automatizada verifican reencolado, agotamiento y reparación.

Evidencias de funcionamiento: recepción y monitoreo

Las figuras 3 a 8 corresponden a capturas directas de la terminal de Visual Studio Code aportadas con el documento. Sus valores dependen de la ejecución y no necesariamente coinciden con los archivos de registros obtenidos en otras corridas.

Figura 3

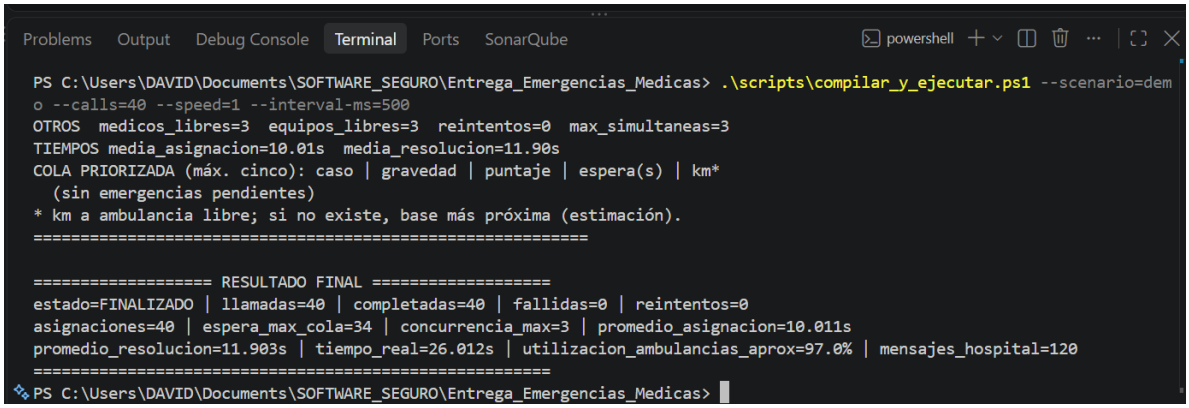
Escenario demo con 30 emergencias completadas



Nota. La captura muestra 30 completadas, ninguna fallida, concurrencia máxima de 3 y espera máxima de 24 casos. La cola aparece vacía porque el monitor fue capturado al finalizar.

Figura 4

Segundo escenario demo con 40 emergencias

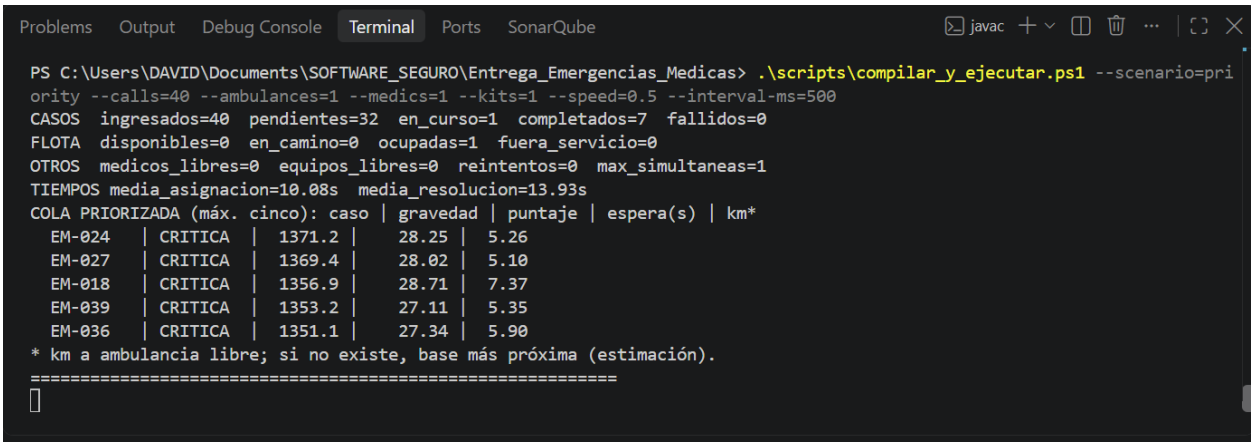


Nota. Resultado de otra corrida: 40 completadas, 0 fallidas y 34 casos como máximo simultáneamente pendientes; el tiempo real observado fue de 26,012 s.

Evidencias: prioridad y recuperación ante fallos

Figura 5

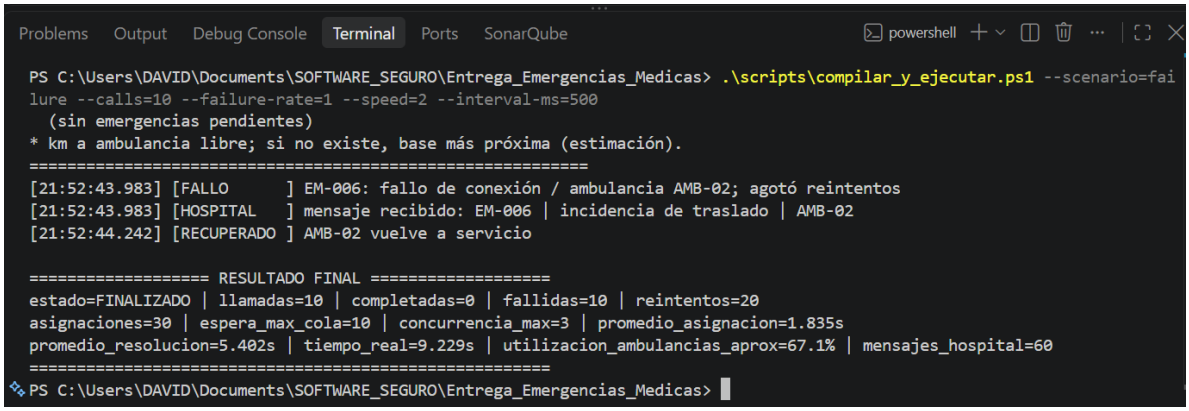
Ejecución del escenario de prioridad con un recurso por clase



Nota. Se observan 20 incidentes completados, 19 pendientes en el máximo de cola y concurrencia máxima de 1. La captura final no muestra el orden de los casos en espera; ese comportamiento se evalúa adicionalmente con pruebas del algoritmo.

Figura 6

Escenario de fallo forzado y recuperación

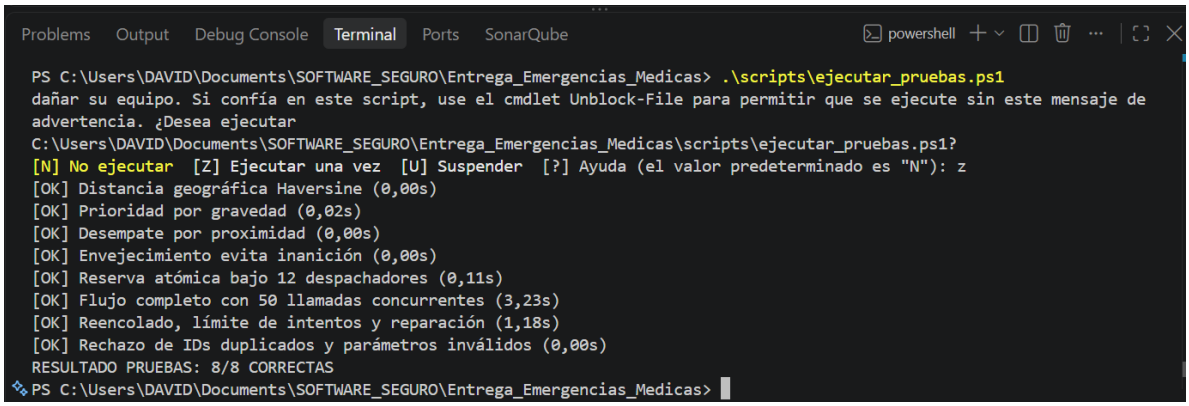


Nota. Con 10 emergencias y failure-rate=1, la consola registra FALLO, un mensaje al HOSPITAL y RECUPERADO; el resultado final contabiliza 10 fallidas y 20 reintentos.

Evidencias: verificación y rendimiento

Figura 7

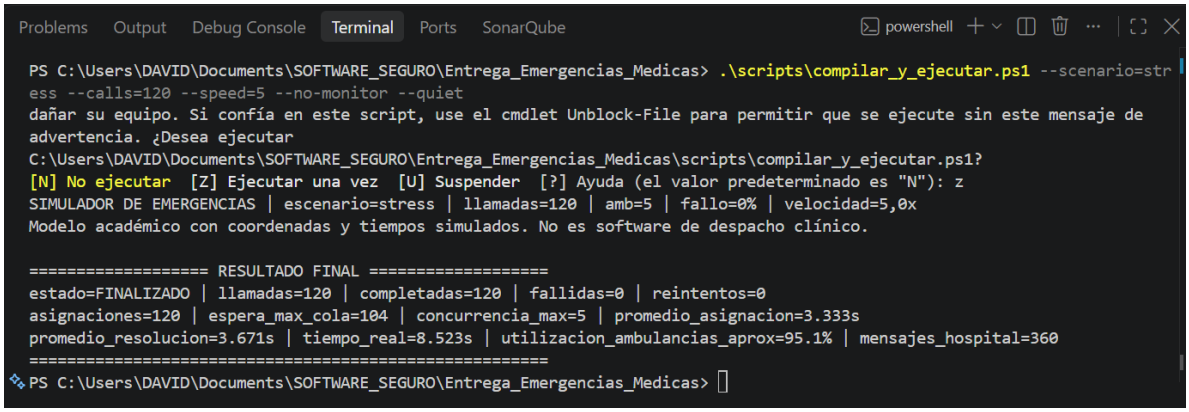
Ejecución de las ocho pruebas automatizadas



Nota. La terminal presenta 8/8 CORRECTAS, entre ellas la exclusión mutua bajo 12 despachadores, el flujo de 50 llamadas, las validaciones y la recuperación ante fallos.

Figura 8

Ensayo de carga con 120 emergencias



Nota. Captura de una corrida de 120 llamadas: 120 completadas, 0 fallidas, concurrencia máxima 5, tiempo total 8,523 s y media de asignación 3,333 s.

Análisis de rendimiento y conclusiones

Las mediciones de la tabla 3 proceden de los archivos evidencias/rendimiento_20.txt, ejecucion_estres_60.txt y rendimiento_120.txt del repositorio. Son registros de corridas independientes de las capturas de terminal; por eso la ejecución de la figura 8 difiere ligeramente en duración.

Tabla 3

Comparación de ejecuciones registradas con cinco ambulancias

Llamadas	Completadas	Duración (s)	Asignación media (s)	Uso aprox. (%)
20	20	2,026	0,494	75,2
60	60	4,493	1,637	92,6
120	120	8,649	3,382	94,2

Al aumentar de 20 a 120 casos con una flota fija de cinco ambulancias, el tiempo total pasó de 2,026 a 8,649 segundos y la espera media de asignación de 0,494 a 3,382 segundos. La utilización aproximada aumentó al 94,2 %. Este comportamiento es consistente con la saturación progresiva de recursos en la simulación; las cifras no son tiempos clínicos reales.

Conclusiones y limitaciones

La arquitectura distingue recepción, priorización, despacho, traslado y notificación. La reserva atómica evita adjudicaciones simultáneas del mismo paquete en los escenarios evaluados. Las ocho pruebas automatizadas y las corridas de carga respaldan la funcionalidad observada, sin constituir una garantía formal para toda ejecución posible. Entre las mejoras futuras están la persistencia de incidentes, comunicaciones durables, métricas reproducibles, gestión explícita de entradas inválidas y validación clínica.

Disponibilidad del código

Repositorio público: https://github.com/edalex99/ENTREGA_EMERGENCIAS_M-DICAS

Referencias

Caso de estudio - Sistema de gestión de emergencias médicas. (s. f.). [Material académico no publicado].

S30 - EA2: Sistema de gestión de emergencias médicas con concurrencia. (s. f.). [Guía de actividad académica no publicada].